

C++/Qt Desktop Application

1. Tổng quan về dự án

Phát triển một ứng dụng Windows Desktop sử dụng C++ và Qt, cung cấp các tính năng quản lý File/Folder, tìm kiếm, giám sát luồng (thread) của ứng dụng và quản lý giấy phép (license).

Ứng dụng cần được thiết kế theo kiến trúc mô-đun để mỗi chức năng chính có thể được phát triển và kiểm thử một cách độc lập.

Hệ thống bao gồm:

a. Main Application

- Cung cấp giao diện người dùng.
- Xử lý các thao tác với File/Folder.
- Thực hiện chức năng tìm kiếm.
- Giám sát các luồng của ứng dụng.
- Giao tiếp với License Server.

b. License Server Application

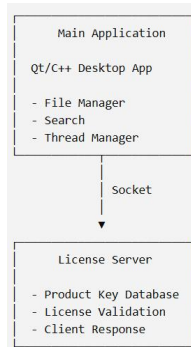
- Chạy như một ứng dụng độc lập.
- Lưu trữ và quản lý các Product Key hợp lệ.
- Tiếp nhận các yêu cầu xác thực giấy phép từ Main Application.
- Giao tiếp với Main Application thông qua cơ chế Socket.

2. License Management

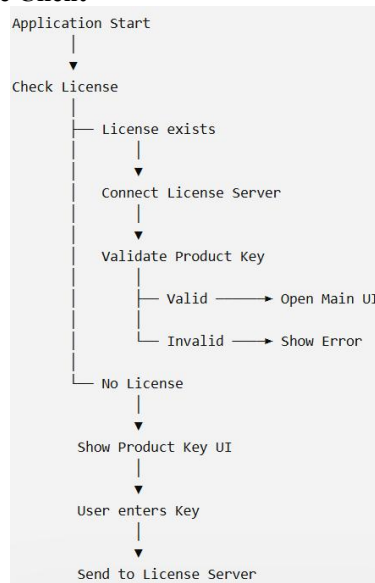
Mô-đun này cần được triển khai trước tiên vì ứng dụng chính cần xác thực giấy phép trước khi cho phép người dùng truy cập vào các chức năng chính của ứng dụng.

2.1 License Architecture

Hệ thống bao gồm hai ứng dụng độc lập:



2.2 Main Application — License Client



2.3 Product Key

Main Application gửi Product Key đến License Server thông qua socket. Server kiểm tra key và phản hồi lại.

2.4 License Server

License Server cần duy trì danh sách các Product Key hợp lệ. Và có nhiệm vụ:

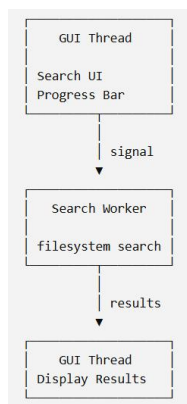
- Chấp nhận kết nối từ máy khách (client).
- Tiếp nhận các Product Key.
- Xác thực các Product Key.
- Trả về kết quả xác thực.
- Quản lý các giấy phép đã lưu trữ.

3. File & Folder Management

Các chức năng :

- Open.
- Rename.
- Delete.
- Copy.
- Move.
- Create.
- Properties.
- Upload

4. Search



Tránh đơ/treo giao diện (GUI freezes): ví dụ chạy tác vụ duyệt 1 lượng lớn file trên GUI Thread (luồng chính xử lý giao diện), luồng này bị chiếm dụng hoàn toàn. Giao diện ứng dụng sẽ "đứng hình" (Not Responding), người dùng không thể nhấn nút, cuộn trang hay thao tác khác.

Tối ưu trải nghiệm người dùng (UX): Khi tách sang Worker Thread (luồng phụ), GUI Thread tiếp tục rảnh rỗi để duy trì hiệu ứng mượt mà, cho phép hiển thị thanh tiến trình (Progress Bar), nút "Hủy tìm kiếm" (Cancel) hoặc để người dùng làm việc khác trong app.

Tận dụng đa nhân CPU: Giúp hệ thống phân chia công việc xử lý I/O đĩa cứng (filesystem search) cho một luồng riêng, không làm ảnh hưởng đến luồng về giao diện.

Cơ chế giao tiếp bất đồng bộ (Asynchronous):

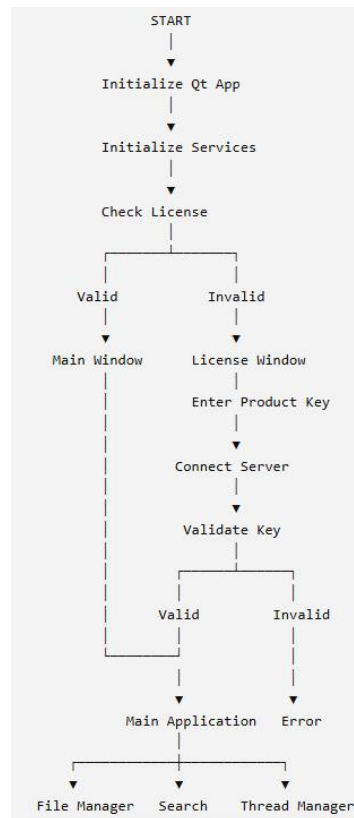
- GUI Thread gửi tín hiệu (signal) yêu cầu tìm kiếm sang Search Worker.
- Search Worker âm thầm (theo tên file, tìm kiếm đệ quy,...) chạy ngầm dưới đĩa cứng.
- Sau khi hoàn thành, Worker trả kết quả (results) về cho GUI Thread để hiển thị lên màn hình (Display Results).

5. Process Management

Xử lý chia các task nào quan trọng làm trước, ít quan trọng làm sau. Nâng cao hiệu suất của app và tránh các tình trạng xung đột và xử lý các ngoại lệ.

- License Management (Priority 4) - Mức cao nhất
- File & Folder Management (Priority 3)
- Search (Priority 2)
- Process Management (Priority 1) - Mức thấp nhất

6. Application flow



7. Technology

Component	Technology
Programming language	C++
GUI	Qt
Build System	CMake
Compiler	MinGW
Networking	Qt TCP socket
Licence storage	SQLite
Architecture	Client - Server

8. Một số function khác:

- Sử dụng các thuật toán mã hóa như AES hay SSL/TLS trên QTcpSocket để bảo vệ khóa Public key và dữ liệu sẽ được an toàn hơn.
- Áp dụng một số thuật toán tìm kiếm thông minh hơn trong trường hợp có lượng dữ liệu lớn thay vì các phương pháp tìm kiếm thông thường.
- Thêm file Logging System: Mọi thao tác (đăng nhập license, lỗi file, kết quả tìm kiếm) cần được ghi vào một file .log để hỗ trợ kỹ thuật khi có sự cố.
- Biểu đồ tài nguyên hiển thị phần trăm CPU/RAM mà mỗi thread của ứng dụng đang chiếm dụng dưới dạng đồ họa dễ trông trực quan hơn.
- Một số phím tắt cơ bản để người dùng sử dụng tiện hơn như Ctrl+C, Ctrl+V , Ctrl+X,...